

Morris Traversal (Pre order)

N L R

Algo:

curr = root

```
while (curr != NULL) {  
    if (curr → left) {  
        pred = curr → left  
        while (pred → right) {  
            pred = pred → right;  
        }  
        pred → right = curr → right  
        curr → right = curr → left  
        curr → left = NULL;  
    }
```

curr = curr → right

}

// left part NULL

curr = root;

while (curr != NULL)

11/6/69 Binary Search Tree

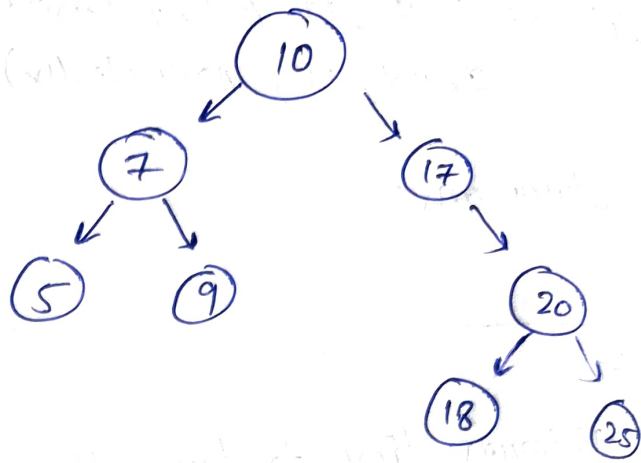
→ left child is less than parent

→ right child is greater than parent

NOTE: for a node (entry)

All nodes in left subtree should be less & right subtree greater.

NOTE: Inorder of BST is always sorted.



Implementation:

Class Node {

public:

int data;

Node* left;

Node* right;

Node (int d) {

this->data = d;

this->left = NULL;

this->right = NULL;

}

void takeInput (Node* &root) {

int data;

cin >> data;

while (data != -1) {

root = insertIntoBST (root, data);

cin >> data;

}

}

Node* insertIntoBST (Node* root,
int d) {

if (root == NULL) {
Node* root = new Node(d);
return root;
}

if (d > root->data) {

root->right = insertToBST(
root->right, d);

} else {

root->left = insertIntoBST(
root->left, d);

}

return root;

}

int main () {

Node* root = NULL;

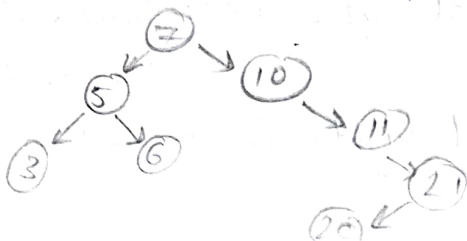
cout << "Enter data to create
BST" << endl;

takeInput (root);

}

i/p → 7, 10, 11, 5, 6, 3, 21, 20.

o/p →



insert of node in BST

↳ T.C = $O(\log n)$.

Search in BST.

TreeNode searchBST (root, x) {

if (root == NULL) {
return NULL;
}

if (x == root->val) {
return root;
}

if (x > root->val) {
return searchBST (root->
right, val);

} else {

return searchBST (root->left,
val);

}

}

Deletion in BST.

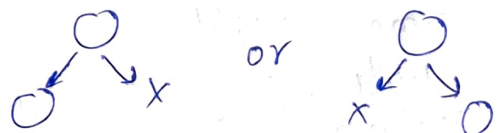
NOTE: min value in BST is always
at leftmost node and max
value at rightmost node.

Node to delete has 3 cases

1) 0 children

↳ directly delete that node
↳ return null.

2) 1 child



$\hookrightarrow$ temp = root $\rightarrow$ child
 delete (root)
 return temp

3) 2 children

$\hookrightarrow$ replace root $\rightarrow$ val with
 min val in right subtree
 OR
 with max val in left subtree.

$\hookrightarrow$ delete replaced leaf node.

code

Node* deleteFromBST (Node* root,
int val) {

if (root == NULL)

return root;

if (root $\rightarrow$ data == val) {

if (root $\rightarrow$ left == NULL &&
root $\rightarrow$ right == NULL) {

delete root;

return NULL;

}

0 children

if (root $\rightarrow$ left != NULL && root $\rightarrow$ right
== NULL) {

Node* temp = root $\rightarrow$ left;

delete root;

return temp;

}

left child

if (== && !=) {

temp = right

delete

return temp;

right child

if (root $\rightarrow$ left != NULL && root $\rightarrow$ right
== NULL) {

int mini = minVal (root $\rightarrow$ right)
 $\rightarrow$ data;

root $\rightarrow$ data = mini;

root $\rightarrow$ right = deleteFromBST(
root $\rightarrow$ right, mini;

return root;

}

T.C = $O(h)$

else if (root $\rightarrow$ data > val) {

root $\rightarrow$ left = deleteFromBST
return root; (root $\rightarrow$ left, val);

}

root $\rightarrow$ right = deleteFromBST(
root $\rightarrow$ right, val);

return root;

}

lec 70

Q.1) Validate BST.

$\rightarrow$ intuition : check range for
each node of BST.

code

bool isBST (root, min, max) {

if (root == NULL) {

return true;

}

bool left = isBST (root $\rightarrow$ left, min,
root $\rightarrow$ val);

bool right = isBST (root $\rightarrow$ right,
root $\rightarrow$ val, max);

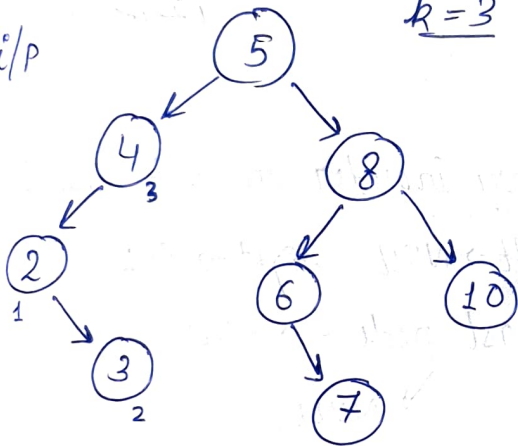
```
bool ok = (root->val > min &&
           root->val < max)
equality xxx
```

```
if (left && right && ok) {
    return true;
} else {
    return false;
}
```

Q.2) Find k^{th} smallest element in BST.

$k=3$

i/p



o/p = 4

intuition: Find k^{th} element in inorder traversal of BST.

code:

```
int solve (root, &i, k) {
    if (root == NULL) {
        return -1;
    }
    // L
    int left = solve (root->left,
                     i, k)
    // N
    if (left != -1) { return left; }
```

```
// N p++
if (i == k)
    return root->data;
```

// R

```
return solve (root->right, i, k);
}
```

T.C = $O(n)$ S.C = $O(h)$

Q.3) Inorder Successor / Predecessor in BST.

Successor = next element in inorder traversal

Predecessor = previous element

approach: ① find key node

② max element in left subtree is predecessor and min element in right subtree is successor.

Q.4) ~~CA~~ LCA (Lowest Common Ancestor) of a BST.

TreeNode <int> * LCA (root, p, q) {

```
if (root == NULL) {
    return NULL;
}
```

```
if (root->data < p->data &&
    root->data < q->data) {
    return LCA (root->right, p, q);
```

if (root → data > p &&
 root → data > q)

return LCA (root → left, p, q);

return root;
 }

uc71

Q.1) Two sum in BST

↳ return T or F

approach 1

Brute force: TC: $O(n^2)$

① visit a node

② search (Target - node → val) in
 left & right subtrees of that
 node. ↳ return true if found.

app²

T.C = $O(n)$

① Take in order traversal

eg 2 6 8 10 12 15
 i j

while (i < j)
 sum = arr[i] + arr[j]

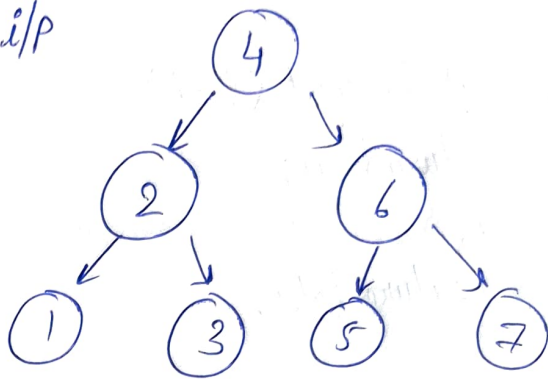
if (sum > target)
 j--;

if (sum < target)
 i++;

if (sum == target) return {i, j}.
 ↳ True.

Q.2) Flatten a BST in sorted
 list.

i/p



o/p ① → ② → ③ → ④ → ⑤
 ⑦ ← ⑥

app

① store inorder in vector <node*>

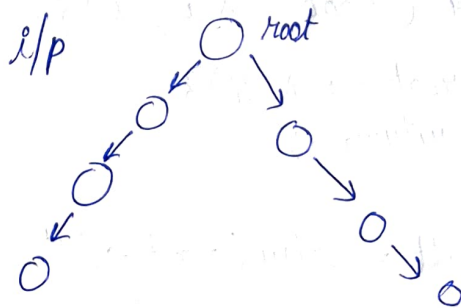
② left → NULL right → i+1

③ last node → NULL
 ↳ NULL.

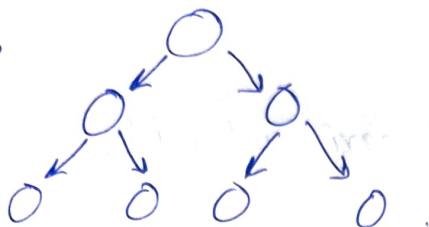
Q.3) Normal BST to Balanced
 BST

→ Balanced → $\text{abs}(\text{left}(h) - \text{right}(h)) \leq 1$

for each node



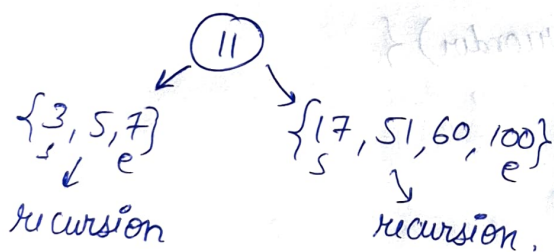
o/p



approach

- ① store inorder
- ② start inserting from mid element or array.
- ③ left wala part left subtree & right wala → right subT.

0	1	2	3	4	5	6	7
3	5	7	11	17	51	60	100
s			m				e



code

```

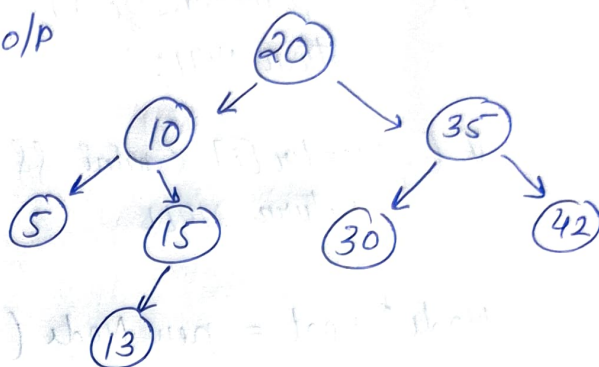
Node* inorderToBST (s, e, TLEinorder) {
    if (s > e) return NULL;
    int mid = (s+e)/2;
    Node* root = new Node (inorder [mid]);
    root->left = inToBST (0, mid-1, inorder);
    root->right = inToBST (mid+1, e, inorder);
    return root;
}

```

Q.4 ** BST from preorder

i/p 20, 10, 5, 15, 13, 35, 30, 42

o/p



approach 1 — $O(n^2)$

↳ Traverse input array and keep inserting each node in BST.

approach 2

↳ preorder given
↓ sort
inorder

↓ $O(N \log N)$

Construct a Binary Tree.

approach 3

$O(n)$

↳ check range for each node and travel preorder to insert.

if ($i < \text{MIN}$ && $i > \text{MAX}$)
return NULL;

code

```
Node* solve (vector<int> preorder, int mini, int maxi, int &i) {  
    if (i >= preorder.size())  
        return NULL;
```

```
    if (preorder[i] < mini || preorder[i] > maxi) → out of range  
        return NULL
```

```
    Node* root = new Node (preorder[i++]);
```

```
    root → left = solve (preorder, mini, root → data, i);
```

```
    root → right = solve (preorder, root → data, maxi, i);
```

```
    return root;
```

```
}
```

```
Node* preorderToBST (vector<int> preorder) {
```

```
    int mini = INT_MIN;
```

```
    int maxi = INT_MAX;
```

```
    int i = 0;
```

```
    return solve (preorder, mini, maxi, i);
```

```
}
```

LC 72

Q.1) Merge 2 BST ***

o/p



approach 1:

① BST 1 BST 2
 (inorder 1) (inorder 2)

② merge two sorted arrays

③ arr 3 → BST

↳ return

T.C → $O(m+n)$

S.C → $O(m+n)$

code 1

```
void inorder (root, vector & in) {  
    .....  
}
```

~~Node * merge BST~~

```
vector mergeArr (vector1, v2) {  
    vector <int> ans;  
    int i=0, j=0;  
    while (i < a.size() && j < b.size())  
        if (a[i] < b[j])  
            ans[k++] = b[j];  
            j++;  
        else  
            ans[k++] = a[i];  
            i++;  
    }  
}
```

~~Node *~~

```
Node * inorderToBST (vector & in) {  
    .....  
}
```

```
Node * mergeBST (root1, root2) {
```

```
    vector bst1, bst2;  
    inorder (root1, bst1);  
    inorder (root2, bst2);
```

```
    vector <int> mergeArray =  
        mergeArrays (bst1, bst2);
```

```
    int s=0, e = mergeArray.size()-1;  
    return inorderToBST (s, e, mergeArray);  
}
```

approach 2

T.C = $O(m+n)$
B.C = $O(h_1 + h_2)$

① root1 root2
 ↓ ↓
sorted LL1 LL2 (sorted)

② merge two sorted linked list.

③ sorted linked list → BST (return)

code

uc73

Q.:- Largest BST in Binary Tree.

→ return size.

approach 1: visit every node

and check valid BST of

T.C = $O(n^2)$

store
max size

approach 2:

node

→ chuck 3 things (stored values)

1) left subtree is BST

2) right subtree is BST

3) $\text{max. left} < \underset{\substack{\uparrow \\ \text{data}}}{\text{root}} < \text{min. right}$

for

Each node we will store 4 things (subtree).

{ max , min , bool , size }

↓
true/false

code

class info {

public :

```
int maxi;
```

```
int mini;
```

bool isbst;

```
int size;
```

3.

```
info solve (Node* root, int &ans) {
```

```
if (root == NULL) {
```

```
return { INT_MIN, INT_MAX,
        true, 0 };
```

3

info left = solve (root $\rightarrow$ left,
ans);

info right = solve (root $\rightarrow$ right,
ans);

```
info currNode;
```

$$\text{currNode.size} = \text{left.size} + \text{right.size} + 1;$$
$$\text{currNode.maxi} = \max(\text{root} \rightarrow \text{data}, \text{root} \rightarrow \text{right.maxi})$$
$$\text{currNode.mini} = \min(\text{root} \rightarrow \text{data}, \text{left.mini});$$

if (left.isBST && right.isBST
 && (root->data > left.maxi
 && root->data < right.min))

currNode.isBST = True;

```
} else {
```

```
currNode.isBST = false;
```

3

if (curr Node. is BST) {

ans = max (ans, currNode.
size);

}

```
return currNode;
```

3

```

int largestBST (Node* root) {
    int maxSize = 0;
    info temp = solve(root, maxSize);
    return temp.maxSize;
}

```

MC 74

HEAP

→ Complete Binary Tree which follows heap order property.

CBT

level

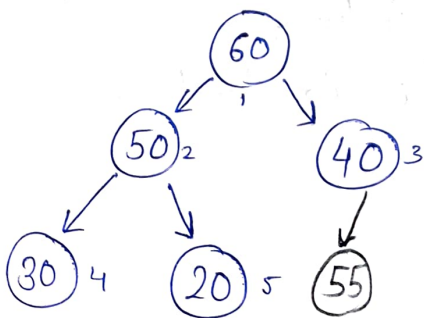
→ every tree is completely filled except last level.

→ nodes are always added from left.

• max heap → child < parent
 • min heap → child > parent.

Insertion in Heap (max)

→ implementing using arrays.



val = 5
insert

X	60	50	40	30	20	55
0	1	2	3	4	5	6

NOTE: i^{th} Node ($\pm$ based in)

- 1) left child = $2i$
- 2) right child = $2i+1$
- 3) parent = $i/2$

steps

- 1) insert at end.
- 2) Take it to the write position.

code

Class heap {

int arr[100];

int size = 0;

Public:

void insert (int val) {

size++;

int index = size;

arr[index] = val;

while (index > 1) {

int parent = index/2;

if (arr[parent] < arr[index]) {

swap (arr[parent], arr[index]);

index = parent;

} else {

return;

}

}

T.C = $O(\log n)$

Deletion T.C = $O(\log n)$

↳ of root node

1) Put value of last node in root node

$$a[1] = a[n]$$

2) Remove last node

3) Adjust root node to its correct position.

```
void delete() {
```

```
    if (size == 0) {
```

```
        return;
```

```
    }
```

```
    arr[1] = arr[n];
```

```
    size--;
```

```
    int i = 1;
```

```
    while (i < size) {
```

```
        int leftIndex = 2 * i;
```

```
        int rightIndex = 2 * i + 1;
```

```
        if (leftIndex < size && arr[i] < arr[leftIndex])
```

```
        { swap(arr[i], arr[leftIndex]);
```

```
          i = leftIndex;
```

```
        } else if (rightIndex < size &&
```

```
        arr[rightIndex] > arr[i]) {
```

```
            swap(arr[i], arr[rightIndex]);
```

```
            i = rightIndex;
```

```
        } else {
```

```
            return;
```

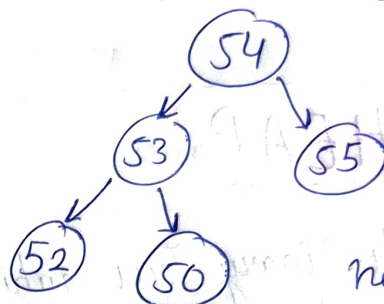
```
    }
```

Heapify (say max heapify)

↳ convert given array into heap array

i.p =

x	54	53	55	52	50
0	1	2	3	4	5



not heap

NOTE:

All leaf nodes in CBT are present at index.

$$\left(\frac{n+1}{2}\right) \text{ to } (n)$$

All leaf nodes are already a heap. So process only 1 to $\frac{n}{2}$ index.

"Ek node ko uske sahi jagah lke jaana heapify function ka kaam hai".

↳ pass index of that node as 'i' in function

Code

```
void heapify(int arr[], int size,
             int i){
```

```
    int largest = i;
```

```
    int left = 2*i;
```

```
    int right = 2*i+1;
```

```
    if (arr[largest] < arr[left]
        && largest < n)
```

```
        largest = left;
```

```
    if (arr[largest] < arr[right]
        && right < n)
```

```
        largest = right;
```

```
    if (largest != i) {
```

```
        swap(arr[largest], arr[i]);
```

```
        heapify(arr, n, largest);
```

```
    }
```

```
}
```

```
int main() { // declare arr & n;
```

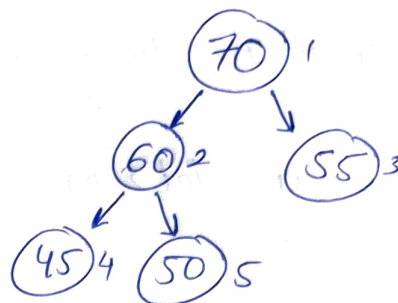
```
    for (int i = n/2; i > 0; i--) {
```

```
        heapify(arr, n, i);
```

```
    }
```

```
}
```

Heap Sort (max Heap)



i/p

x	70	60	55	45	50
0	1	2	3	4	5

algo

1) swap $a[1]$ & $a[n]$

2) size--;

repeat.

3) Root node $\rightarrow$ correct place (using heapify).

4) return arr.

code

```
void heapSort(int arr[], int n){
```

```
    int size = n;
```

```
    while (size > 1) {
```

```
        swap(arr[size], arr[1]);
```

```
        size--;
```

```
        heapify(arr, size, 1);
```

```
    }
```

T.C = $O(n \log n)$

Priority Queue

→ Top element is always max^m

// max heap

```
priority_queue<int> pq;
```

```
pq.push(10);
```

```
pq.push(5);
```

```
pq.top() → 10 {10, 5}
```

```
pq.pop() → 10 {5}
```

// min heap

```
priority_queue<int, vector<int>,  
greater<int>> minheap;
```

lec 75

Q.2) Kth smallest Element

approach 1

① Sort array ($O(n \log n)$)

② return arr[k-1]
'0' based indexing

approach 2

① Build max heap from first k elements. (0 to k-1)

② for rest elements, if (k to n-1)

elements < heap.top()

→ heap.pop()

→ heap.push(element)

③ after processing all elements ans will be heap.top().

Q.2) Check Binary Tree is Heap or not?

→ return true or false.

Algo

① is CBT && is MaxHeap

return true

else
return false.

code

```
bool isCBT(struct Node* root,  
int index, int cnt) {
```

```
if (root == NULL) {  
    return true;  
}
```

```
if (index >= cnt) {  
    return false;
```

```
} else {
```

```
bool left = isCBT(root->left,  
2*index+1, cnt);
```

```
bool right = isCBT(root->right,  
2*index+2, cnt);
```

```
return (left && right);  
}
```

```
}
```

```
bool isMaxOrder (Node* root){
```

```
    if (root → left == NULL &&  
        root → right == NULL)
```

```
        return true;
```

```
    if (root → right == NULL) {
```

```
        return (root → data >  
                root → left → data);
```

```
    } else {
```

```
        bool bool left = isMaxOrder(  
            root → left);
```

```
        bool right = isMaxOrder(  
            root → right);
```

```
        if (left && right &&
```

```
            root → d > root → l → d &&  
            root → d > root → r → d) {
```

```
            return true;
```

```
        }  
        else {
```

```
            return false;
```

```
        }
```

```
    }
```
